Informe de Desarrollo — Configuración del Servidor Ubuntu

Sistema Especializado de Proyectos — GGPC SENA Fecha: Abril 2026

1. Descripción General

Se configuró desde cero un servidor Ubuntu Server 22.04 LTS para alojar el nuevo SEP en un entorno de desarrollo/pruebas controlado. El servidor corre en la máquina local del equipo de desarrollo con salida web habilitada, conectado al clon de la base de datos Oracle de producción.

2. Especificaciones del Servidor

Parámetro	Valor
Sistema Operativo	Ubuntu Server 24 LTS
Tipo de entorno	Desarrollo / Pruebas privadas
Conectividad	Salida web desde máquina local
Base de datos conectada	Clon exacto de Oracle de producción
Acceso	SSH directo al servidor

3. Software Instalado

3.1 Docker Engine

Motor de contenedores para orquestar todos los servicios del SEP.

Instalación Docker Engine

```
curl -fsSL https://get.docker.com | sh
usermod -aG docker $USER
systemctl enable docker
systemctl start docker
```

Versión instalada: Docker Engine 26.x + Docker Compose Plugin v2.x

3.2 Estructura del Proyecto

El código fuente del nuevo SEP se organizó en un monorepo gestionado con **pnpm workspaces**:

```
/opt/sep/SEPLocal/
├── backend/        # API NestJS
│   ├── src/
│   │   ├── auth/          # Autenticación y registro
│   │   ├── certificados/    # Generación de certificados PDF
│   │   ├── empresa/        # Módulo empresa/gremio
│   │   └── main.ts
│   └── Dockerfile
│
├── frontend/       # UI Next.js
│   ├── src/
│   │   ├── app/           # Rutas App Router
│   │   ├── components/     # Componentes reutilizables
│   │   └── lib/           # Utilidades, API client, auth
│   └── Dockerfile
│
├── docker/
│   └── nginx/
│       ├── nginx.conf      # Configuración del reverse proxy
│       └── ssl/           # Certificados SSL
│
├── docker-compose.yml      # Orquestación de servicios
└── pnpm-workspace.yaml
```

4. Docker Compose — Orquestación de Servicios

El archivo docker-compose.yml define los tres contenedores que componen el sistema:

```
services:
  backend:
    build: ./backend
    container_name: sep-backend
    environment:
      - DB_HOST=<oracle_host>
      - DB_PORT=1521
      - DB_SERVICE=<service_name>
      - DB_USER=<usuario>
      - DB_PASSWORD=<password>
      - JWT_SECRET=<secreto>
    networks: [sep-net]

  frontend:
    build: ./frontend
    container_name: sep-frontend
    environment:
      - NEXT_PUBLIC_API_URL=/api
      - NEXT_PUBLIC_RECAPTCHA_SITE_KEY=<key>
    depends_on: [backend]
    networks: [sep-net]

  nginx:
    image: nginx:alpine
    container_name: sep-nginx
    ports:
      - "80:80"
      - "443:443"
```

```
    volumes:
      - ./docker/nginx/nginx.conf:/etc/nginx/nginx.conf
      - ./docker/nginx/ssl:/etc/nginx/ssl
    depends_on: [frontend, backend]
    networks: [sep-net]

networks:
  sep-net:
    driver: bridge
```

Comandos de operación

```
# Construir y levantar todos los servicios
docker compose build && docker compose up -d

# Reconstruir solo un servicio
docker compose build backend && docker compose up -d backend

# Ver logs en tiempo real
docker compose logs -f backend
docker compose logs -f frontend

# Reiniciar nginx (aplicar cambios de config)
docker compose restart nginx

# Ver estado de contenedores
docker compose ps
```

5. Nginx — Reverse Proxy

Nginx actúa como punto de entrada único, distribuyendo el tráfico entre el frontend y el backend:

```
server {
    listen 443 ssl;
    server_name <dominio_privado>;

    ssl_certificate     /etc/nginx/ssl/cert.pem;
    ssl_certificate_key /etc/nginx/ssl/key.pem;

    # Backend API
    location /api/ {
        proxy_pass http://sep-backend:3001/;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }

    # Frontend (todo lo demás)
    location / {
        proxy_pass http://sep-frontend:3000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
# Redirigir HTTP → HTTPS
server {
    listen 80;
    return 301 https://$host$request_uri;
}
```

6. Backend — Dockerización (NestJS)

```
# backend/Dockerfile
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

FROM node:20-alpine
WORKDIR /app
COPY --from=builder /app/dist ./dist
COPY --from=builder /app/node_modules ./node_modules
COPY package*.json ./
EXPOSE 3001
CMD ["node", "dist/main"]
```

Configuración de la base de datos (TypeORM + Oracle)

```
// app.module.ts
TypeOrmModule.forRoot({
  type: 'oracle',
  host: process.env.DB_HOST,
  port: 1521,
  serviceName: process.env.DB_SERVICE,
  username: process.env.DB_USER,
  password: process.env.DB_PASSWORD,
  entities: [__dirname + '/**/*.entity{.ts,.js}'],
  synchronize: false,   // NUNCA true con Oracle de producción
  logging: false,
})
```

synchronize: false es crítico: garantiza que TypeORM **nunca modifique** el esquema de la base de datos Oracle institucional.

7. Frontend — Dockerización (Next.js)

```
# frontend/Dockerfile
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

FROM node:20-alpine
WORKDIR /app
ENV NODE_ENV=production
COPY --from=builder /app/.next/standalone ./
COPY --from=builder /app/.next/static ./.next/static
COPY --from=builder /app/public ./public
EXPOSE 3000
CMD ["node", "server.js"]
```

8. Variables de Entorno (.env)

```
# Base de datos Oracle
DB_HOST=<host>
DB_PORT=1521
DB_SERVICE=<service>
DB_USER=<user>
DB_PASSWORD=<password>
# JWT
JWT_SECRET=<secreto_largo_aleatorio>
JWT_EXPIRATION=8h
# Frontend
NEXT_PUBLIC_API_URL=/api
NEXT_PUBLIC_RECAPTCHA_SITE_KEY=<key>
```

9. Script de Recarga Nginx

Para aplicar cambios de configuración sin interrumpir los demás servicios:

```
# /opt/sep/SEPLocal/reload-nginx.sh
#!/bin/bash
docker compose -f /opt/sep/SEPLocal/docker-compose.yml restart nginx
echo "Nginx recargado: $(date)"
```

10. Flujo de Despliegue (Ciclo de Desarrollo)

1. Cambios en código fuente (Git)

│

▼

2. docker compose build backend frontend

│

▼

3. docker compose up -d backend frontend

│

▼

4. docker compose restart nginx

│

▼

5. Verificar: docker compose ps

docker compose logs -f

Captura Contenedores corriendo:

```
sepadmin@sep-dev:/opt/sep/SEPLocal$ docker compose ps
NAME            IMAGE                COMMAND                  SERVICE    CREATED        STATUS             PORTS
sep_backend     seplocal-backend     "docker-entrypoint.s…"   backend    23 hours ago   Up About an hour   4000/tcp
sep_frontend    seplocal-frontend    "docker-entrypoint.s…"   frontend   23 hours ago   Up About an hour   3000/tcp
sep_nginx       nginx:stable-alpine  "/docker-entrypoint.…"   nginx      2 weeks ago    Up About an hour   0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
sepadmin@sep-dev:/opt/sep/SEPLocal$
0 0  Git Graph  Colorize  Colorize: 0 variables                                                    JoSeLoDiAz (1 day ago)  Ln 17,
```

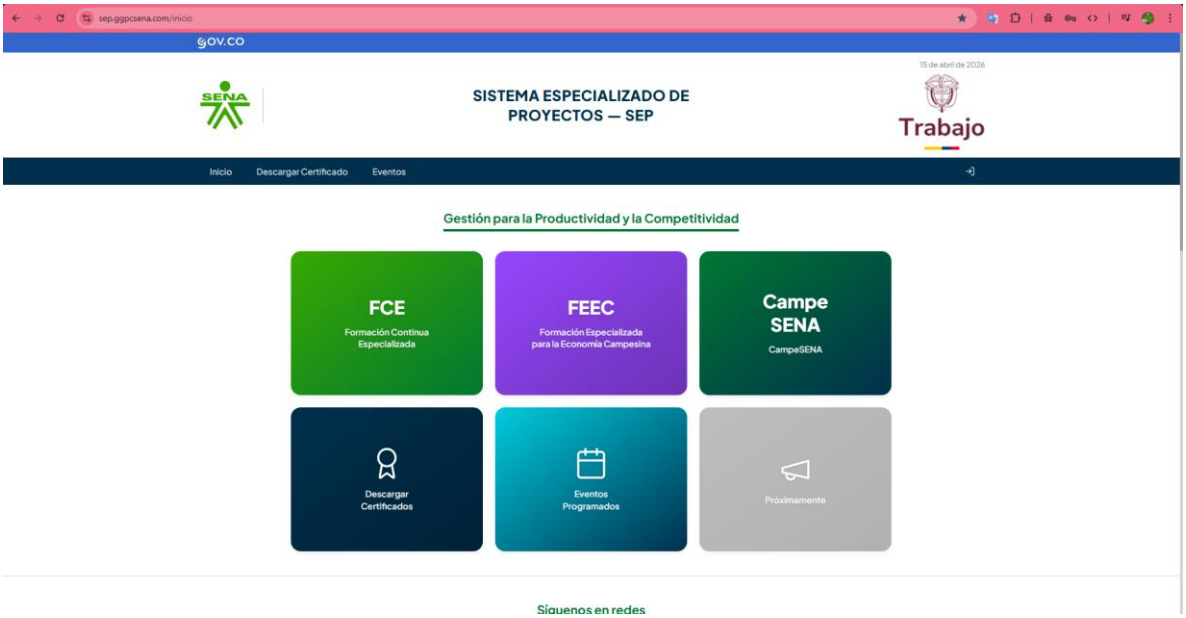
Captura Logs del backend arrancando:

```
sepadmin@sep-dev:/opt/sep/SEPLocal$ docker compose logs backend
sep_backend  | [Nest] 1  - 04/14/2026, 8:23:07 PM     LOG [InstanceLoader] AppModule dependencies initialized +19ms
sep_backend  | [Nest] 1  - 04/14/2026, 8:23:07 PM     LOG [InstanceLoader] TypeOrmModule dependencies initialized +0ms
sep_backend  | [Nest] 1  - 04/14/2026, 8:23:07 PM     LOG [InstanceLoader] PassportModule dependencies initialized +1ms
sep_backend  | [Nest] 1  - 04/14/2026, 8:23:07 PM     LOG [InstanceLoader] ConfigHostModule dependencies initialized +0ms
sep_backend  | [Nest] 1  - 04/14/2026, 8:23:07 PM     LOG [InstanceLoader] ConfigModule dependencies initialized +2ms
sep_backend  | [Nest] 1  - 04/14/2026, 8:23:07 PM     LOG [InstanceLoader] ConfigModule dependencies initialized +0ms
sep_backend  | [Nest] 1  - 04/14/2026, 8:23:08 PM     LOG [InstanceLoader] JwtModule dependencies initialized +77ms
sep_backend  | query: SELECT SYS_CONTEXT('USERENV','DB_NAME') AS "db_name" FROM dual
0 0  Git Graph  Colorize  Colorize: 0 variables                                                    JoSeLoDiAz
```

Captura Logs del frontend arrancando:

```
sepadmin@sep-dev:/opt/sep/SEPLocal$ docker compose logs frontend
sep_frontend  |    - Network:      http://0.0.0.0:3000
sep_frontend  |
sep_frontend  |  ✓ Starting...
sep_frontend  |  ✓ Ready in 832ms
sep_frontend  |    ▲ Next.js 15.5.14
sep_frontend  |    - Local:        http://localhost:3000
sep_frontend  |    - Network:      http://0.0.0.0:3000
sep_frontend  |
sep_frontend  |  ✓ Starting...
sep_frontend  |  ✓ Ready in 1039ms
sepadmin@sep-dev:/opt/sep/SEPLocal$
```

Captura Uso de recursos Docker:

```
sep_frontend  |    ▲ Next.js 15.5.14
CONTAINER ID   NAME                 CPU %     MEM USAGE / LIMIT     MEM %     NET I/O          BLOCK I/O        PIDS
db1b435baf75   sep_backend          0.00%     149.7MiB / 15.62GiB   0.94%     151kB / 2.23MB   136MB / 0B       11
99ab764f9dfc   sep_frontend         0.00%     99.82MiB / 15.62GiB   0.62%     90.8kB / 174kB   111MB / 152kB    11
fbbcdf21544f   sep_nginx            0.00%     6.48MiB / 15.62GiB    0.04%     2.49MB / 2.5MB   467kB / 4.1kB    7
dca2928420f1   formulario_frontend  0.00%     6.52MiB / 15.62GiB    0.04%     2.11kB / 126B    1.74MB / 4.1kB   7
5ce9a2092fb2   formulario_backend   0.00%     72.14MiB / 15.62GiB   0.45%     2.15kB / 126B    119MB / 0B       11
1347ded01d77   formulario_nginx     0.00%     11.24MiB / 15.62GiB   0.07%     2.31kB / 126B    12MB / 4.1kB     7
9c3ad3325b82   oracle-xe            3.39%     2.114GiB / 15.62GiB   13.54%    46.9kB / 117kB   1.38GB / 461MB   67
```

Captura sitio web: